

Sistema de Control de Órdenes de Servicio de un Taller Automotriz

1. Análisis del sistema

1.1 Requisitos funcionales del sistema

- Registrar órdenes utilizando una colección dinámica.
- Guardar número de orden, propietario, placa, servicio y costo.
- Evitar números de orden repetidos.
- Validar textos vacíos y costos menores o iguales que cero.
- Consultar todas las órdenes.
- Buscar una orden por su número.
- Modificar la descripción y el costo.
- Cancelar una orden.
- Consultar todas las órdenes asociadas con una placa.
- Calcular el valor total y el costo promedio.
- Encontrar la orden de mayor costo.
- Mostrar la cantidad de órdenes registradas.
- Controlar errores mediante try-catch.
- Utilizar al menos un bloque finally.
- Mantener el programa funcionando después de controlar un error.

1.2 Clases necesarias y su propósito

Clase	Propósito
-------	-----------

Main	Controla el menú, las entradas y la presentación de resultados.
GestorOrdenes	Administra la colección y las operaciones de las órdenes.
OrdenServicio	Representa y almacena los datos de una orden.
OrdenDuplicadaException	Controla números de orden repetidos.
OrdenNoEncontradaException	Controla operaciones sobre órdenes inexistentes.
DatoInvalidoException	Controla datos que no cumplen las validaciones.

1.3 Atributos de cada clase

Clase: Main

Atributo	Tipo de dato	Visibilidad	Propósito
sc	Scanner	private static final	Leer los datos ingresados por el usuario.
gestor	GestorOrdenes	private static final	Ejecutar las operaciones del sistema.

Clase: GestorOrdenes

Atributo	Tipo de dato	Visibilidad	Propósito
ordenes	List<OrdenServicio>	private final	Almacenar dinámicamente las órdenes activas.

La colección se declara como List<OrdenServicio> y se crea mediante new ArrayList<>().

Clase: OrdenServicio

Atributo	Tipo de dato	Visibilidad	Propósito
numeroOrden	int	private final	Identificar la orden.

nombrePropietario	String	private final	Guardar el nombre del propietario.
placaVehiculo	String	private final	Identificar el vehículo.
descripcionServicio	String	private	Guardar la descripción del servicio.
costoEstimado	double	private	Guardar el costo estimado.

1.4 Métodos de cada clase

Clase: Main

Método	Parámetros	Visibilidad	Propósito
main	args : String[]	public static	Ejecutar el programa y controlar el menú.
mostrarMenu	Ninguno	private static	Mostrar las opciones.
registrarOrden	Ninguno	private static	Solicitar y registrar una orden.
consultarOrdenes	Ninguno	private static	Mostrar todas las órdenes.
buscarOrden	Ninguno	private static	Buscar una orden por número.
modificarOrden	Ninguno	private static	Modificar una orden.
cancelarOrden	Ninguno	private static	Eliminar una orden.
consultarPorPlaca	Ninguno	private static	Mostrar órdenes por placa.
reporteCostos	Ninguno	private static	Mostrar el total y el promedio.

ordenDeMayorCosto	Ninguno	private static	Mostrar la orden más costosa.
cantidadOrdenes	Ninguno	private static	Mostrar la cantidad de órdenes.
leerEntero	mensaje : String	private static	Leer y validar un entero.
leerDouble	mensaje : String	private static	Leer y validar un decimal.

Clase: GestorOrdenes

Método	Parámetros	Visibilidad	Propósito
GestorOrdenes	Ninguno	public	Inicializar el ArrayList.
registrarOrden	Datos de la orden	public	Validar y agregar una orden.
consultarOrdenes	Ninguno	public	Devolver las órdenes registradas.
buscarOrden	numeroOrden : int	public	Buscar una orden.
modificarOrden	Número, descripción y costo	public	Actualizar una orden.
cancelarOrden	numeroOrden : int	public	Eliminar una orden.
consultarPorPlaca	placa : String	public	Buscar todas las coincidencias.
calcularValorTotal	Ninguno	public	Sumar los costos.
calcularCostoPromedio	Ninguno	public	Calcular el promedio.
ordenMayorCosto	Ninguno	public	Encontrar la orden más costosa.

cantidadOrdenes	Ninguno	public	Devolver la cantidad de órdenes.
existeOrden	numeroOrden : int	private	Comprobar si un número ya existe.

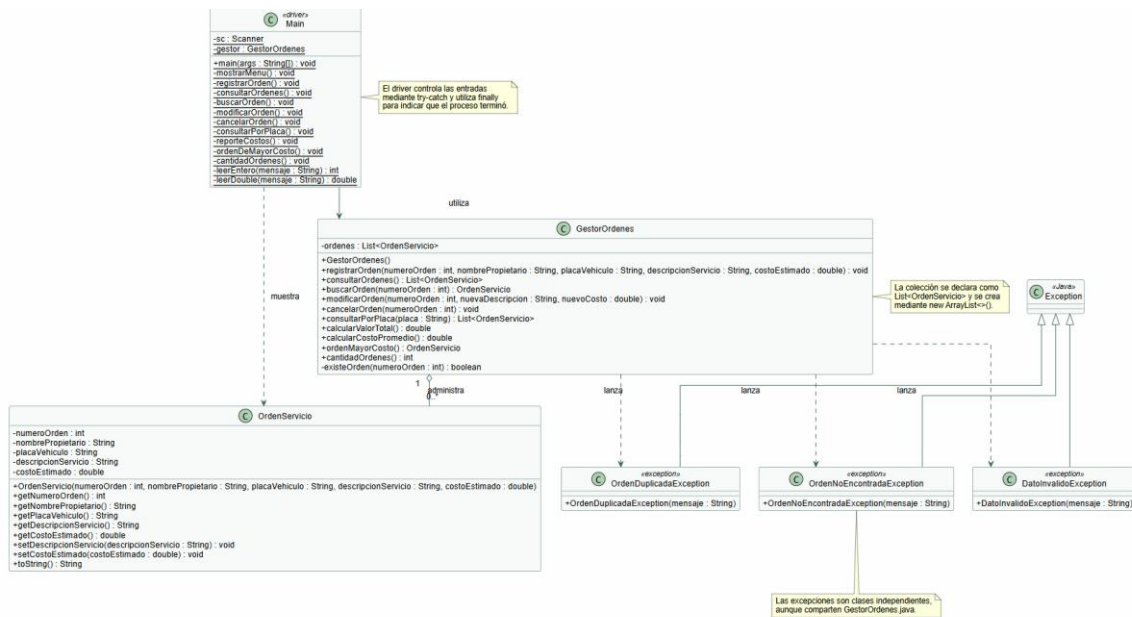
Clase: OrdenServicio

Método	Parámetros	Visibilidad	Propósito
OrdenServicio	Datos de la orden	public	Inicializar una orden.
Métodos get	Ninguno	public	Consultar los atributos.
setDescripcionServicio	descripcionServicio : String	public	Cambiar la descripción.
setCostoEstimado	costoEstimado : double	public	Cambiar el costo.
toString	Ninguno	public	Mostrar la información de la orden.

Excepciones

Clase	Parámetro	Propósito
OrdenDuplicadaException	mensaje : String	Informar que la orden ya existe.
OrdenNoEncontradaException	mensaje : String	Informar que la orden no existe.
DatoInvalidoException	mensaje : String	Informar que un dato no es válido.

2. Diseño: diagrama de clases



Insertar aquí el diagrama UML.

3. Programa

El programa utiliza tres archivos:

- Main.java
- GestorOrdenes.java
- OrdenServicio.java

El menú contiene:

1. Registrar orden.
2. Consultar órdenes.
3. Buscar orden.
4. Modificar orden.
5. Cancelar orden.
6. Consultar órdenes por placa.
7. Reporte de costos.
8. Orden de mayor costo.

9. Cantidad de órdenes.

10. Salir.

GitHub:

<https://github.com/eldeivid68/Ejercicio-3.-Arreglos-din-micos>

Checklist antes de entregar

- El análisis está completo.
- El diagrama incluye atributos, métodos, visibilidad y relaciones.
- El driver program está incluido.
- El código fue subido a GitHub.
- La URL del repositorio fue agregada.